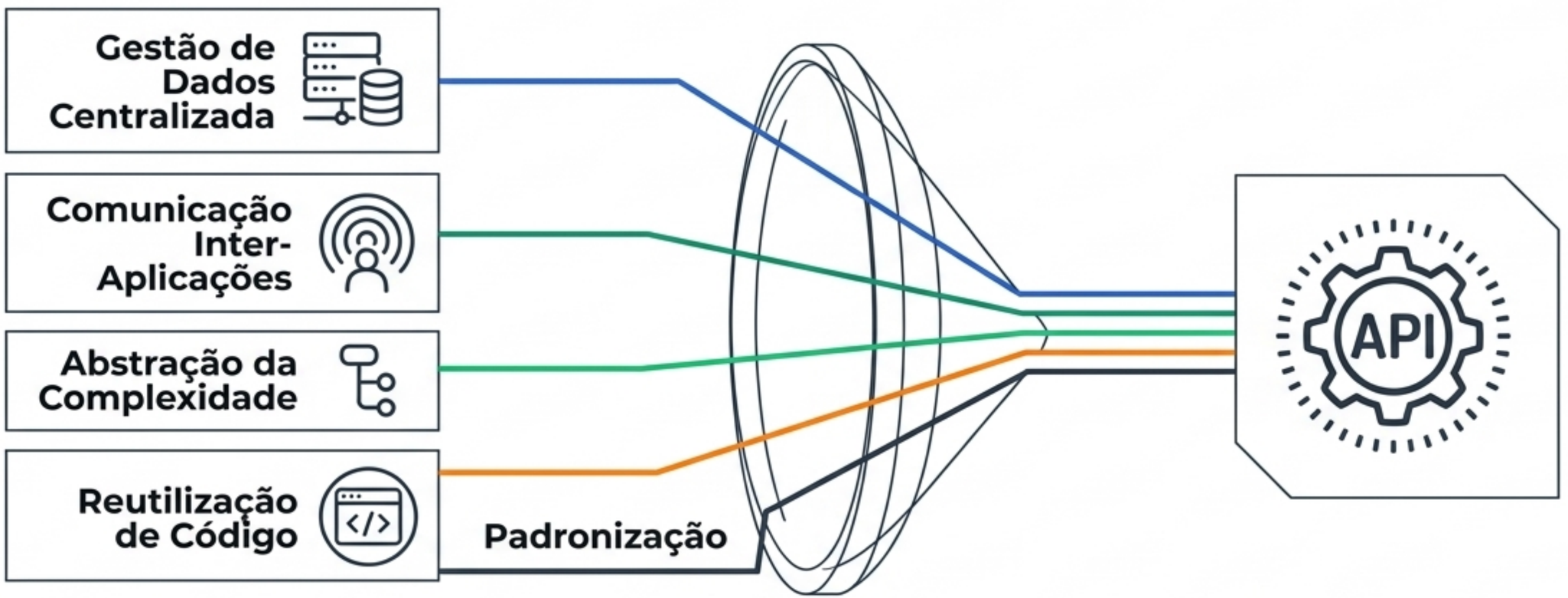


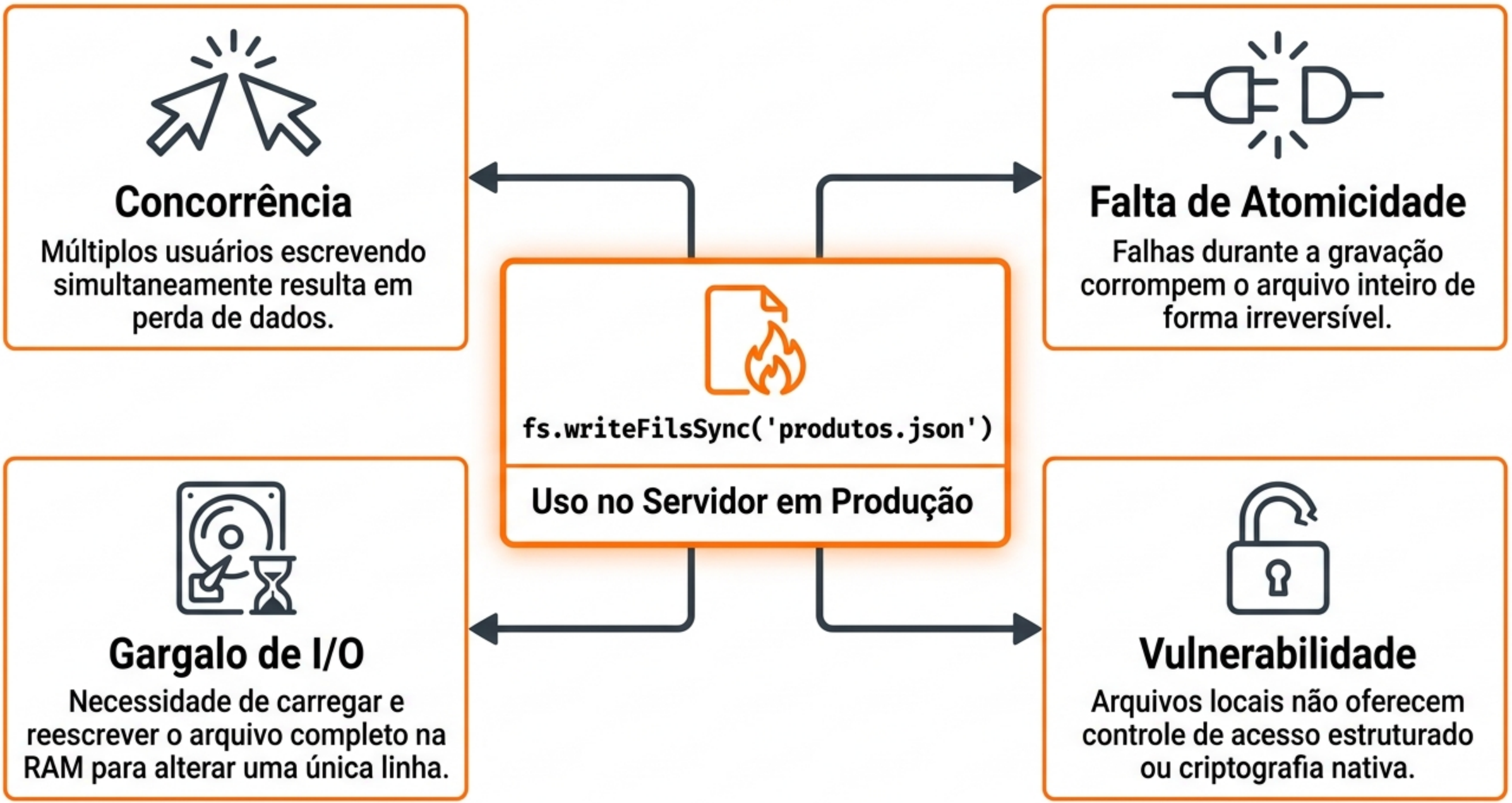
INSTITUTO FEDERAL
DO PIAUÍ

Aula 5.2: API REST + Banco de Dados

Da Persistência Básica à Integração de Alta Performance em Node.js.

Prof. Ronaldo Borges

Gemini Notebook



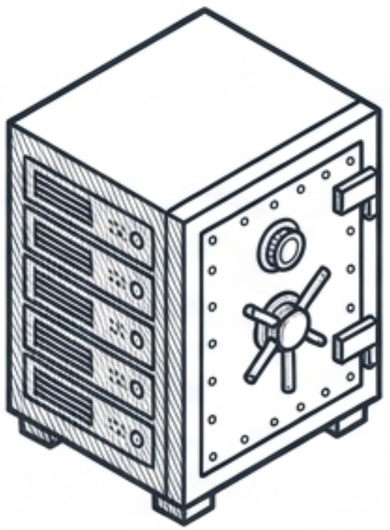
Gemini Notebook

Controle Transacional
(Previne arquivos corrompidos).

Integridade Referencial
(Garante relações lógicas).

Instância Express API

Instância Express API



Instância Express API

Instância Express API

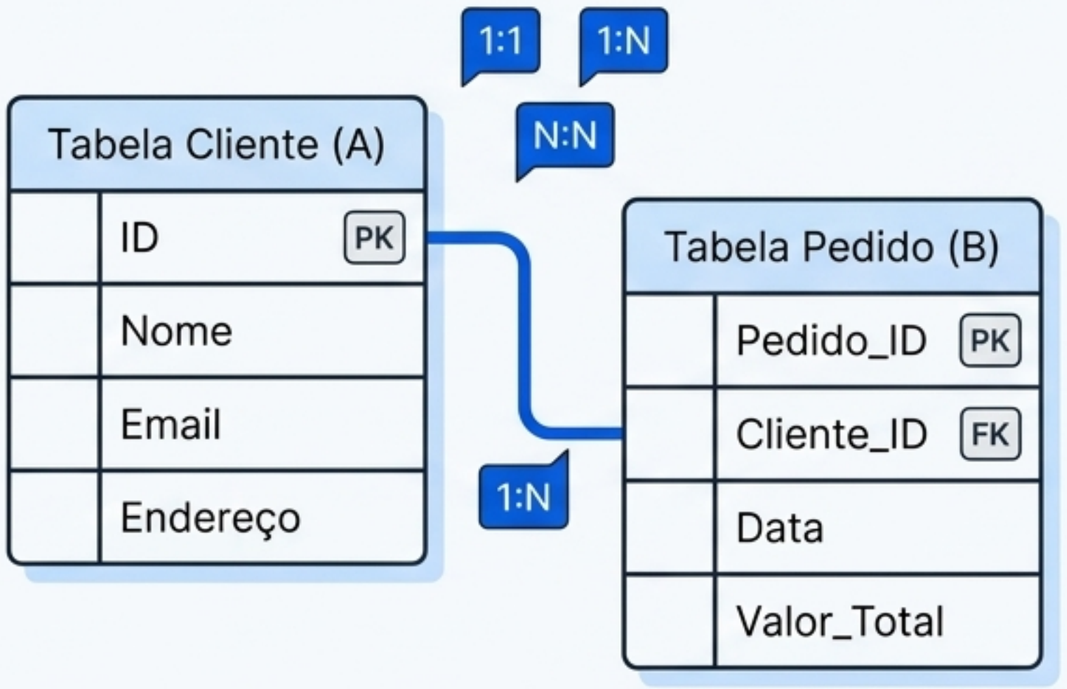
SGBD
Sistema Gerenciador de Banco de Dados

Múltiplas Conexões
(Resolve o gargalo de concorrência).

Indexação Avançada
(Busca em milissegundos sem varrer o disco inteiro).

Gemini Notebook

O Modelo Lógico Relacional Relacional



O Padrão ACID

[A] **[A]tomicidade"**: "Tudo ou Nada." Transações incompletas são imediatamente descartadas.

[C] **[C]onsistência"**: "Regras mantidas." Dados só entram se respeitarem estritamente o esquema.

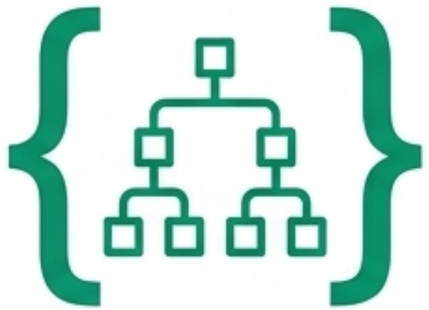
[I] **[I]solamento"**: "Fila invisível." Múltiplas operações simultâneas não interferem entre si.

[D] **[D]urabilidade"**: "Salvo para sempre." A gravação resiste até a falhas de energia.

PostgreSQL | MySQL | SQLite

Gemini Notebook

Paradigmas Não-Relacionais (NoSQL)



Documentos (JSON/BSON)

Foco em dados hierárquicos e esquemas flexíveis (schemaless). Os dados são aninhados.

Ex: MongoDB



Chave-Valor

Foco em acesso ultra-rápido em memória. Usado primariamente para gerenciamento de sessões e Cache.

Ex: Redis



Grafos / Família de Colunas

Foco em relacionamentos complexos, análise de redes sociais, IoT e Big Data massivo distribuído.

Ex: Neo4j, Cassandra

Gemini Notebook

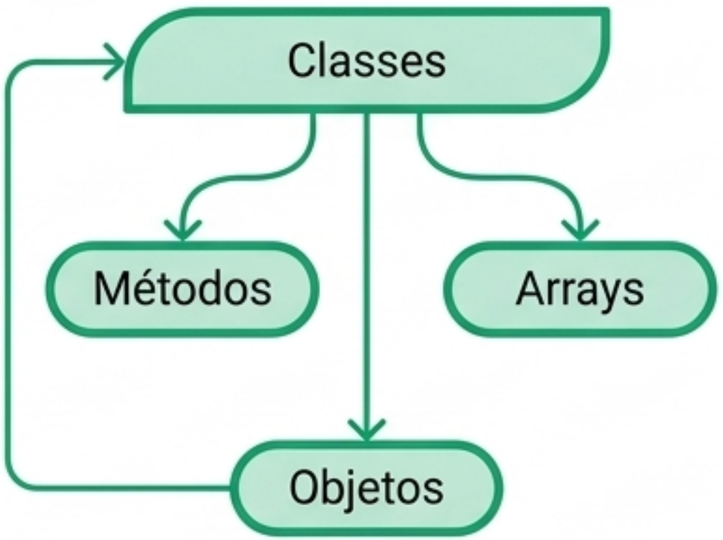
Matriz de Decisão Arquitetural

	SQL (Relacional)	NoSQL (Não-Relacional)
Estrutura de Dados	Esquema rígido (Tabelas, Linhas e Colunas estritas).	Esquema flexível (Documentos e coleções variáveis).
Escalabilidade Principal	Vertical (Adicionar mais CPU/RAM ao mesmo servidor).	Horizontal (Distribuir o banco em múltiplos servidores baratos).
Consistência	Estrita e Imediata (Garantias ACID inegociáveis).	Eventual (Foco em disponibilidade e tolerância à partição).
Cenário Ideal	Sistemas financeiros, ERPs, regras de negócio complexas.	Catálogos rápidos, logs massivos, IoT, redes sociais.

Gemini Notebook

O Problema do Impedance Mismatch

Mundo Node.js / Orientado a Objetos



Linguagem fluida e orientada a objetos (JavaScript / TypeScript).

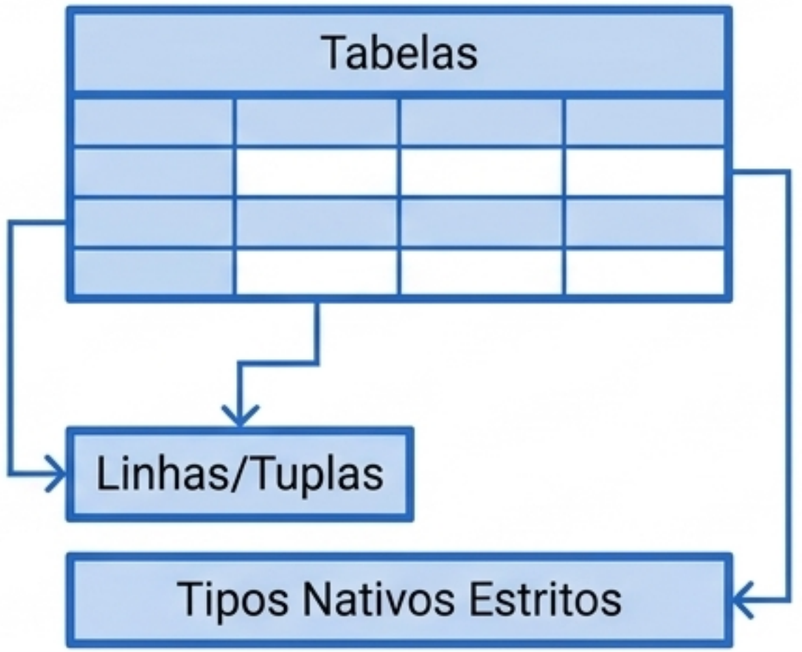
O Problema do Impedance Mismatch

Diferença conceitual grave.

Associações de código não são Chaves Estrangeiras.

O JavaScript não compreende SQL nativamente.

Mundo Relacional / SQL



Restrições matemáticas e esquemas estáticos.

Gemini Notebook

A Ponte de Abstração: ORM e ODM

Antes (Driver Nativo / SQL Puro)

```
SELECT * FROM users
WHERE email = req.body.email;
```



Risco iminente de SQL Injection sem queries parametrizadas. Código Node.js sujo e difícil de manter.

Depois (Abstração via ORM/ODM)

```
User.findOne({ email: req.body.email });
```



Código orientado a objetos. Mitigação nativa de Injeção SQL. Validações integradas.

Prós	Contras
Produtividade massiva, segurança por padrão, tipagem limpa.	Curva de aprendizado inicial, sobrecarga de performance em relatórios analíticos com JOINs hipercomplexos.

Gemini Notebook

Ecossistema de Ferramentas Node.js

Categoria: ORMs para Relacional (SQL)

Prisma

Schema declarativo moderno, tipagem forte automática para TypeScript. O padrão atual recomendado.

Sequelize

Clássico do ecossistema Node. Baseado nativamente em **classes** tradicionais com amplo suporte a diversos SGBDs.

Menção honrosa: TypeORM

Categoria: ODMs para Não-Relacional (NoSQL)

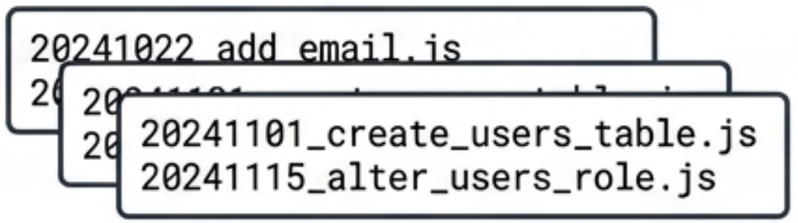
Mongoose

A ferramenta padrão-ouro para o MongoDB no Node.js. Adiciona validação de schema estrita em um banco que originalmente é schemaless.

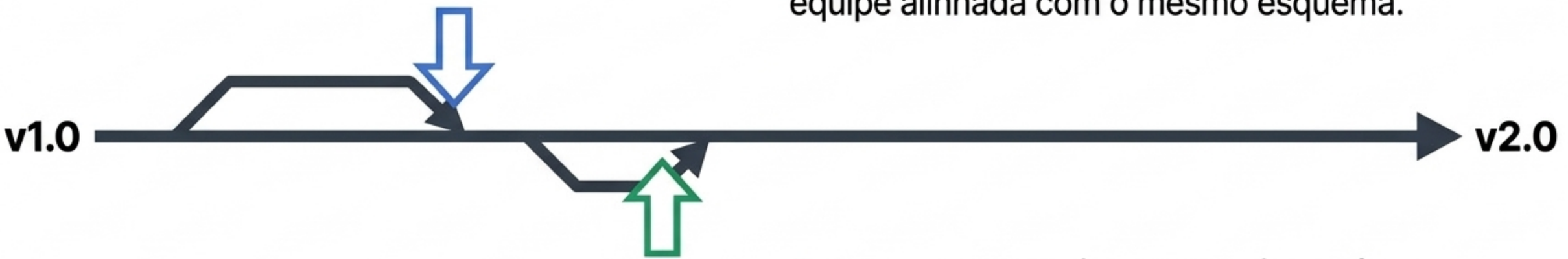
Gemini Notebook

Evolução e Versionamento do Banco

Migrations



Versionamento da estrutura do banco (CREATE, ALTER TABLE). Garante rastreabilidade, previne quebras entre os ambientes de Desenvolvimento e Produção, e mantém toda a equipe alinhada com o mesmo esquema.



Seeds

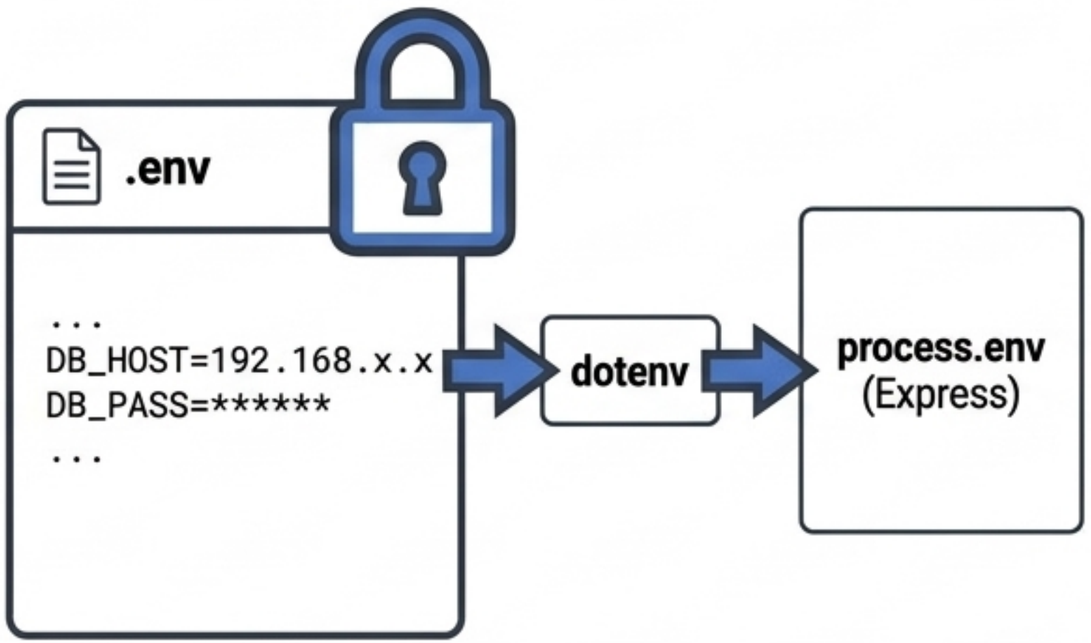


Scripts automatizados executados após as *migrations*. Servem para popular o banco de dados vazio com registros essenciais (ex: usuário admin inicial, categorias base) ou gerar massa de dados para testes automáticos.

Gemini Notebook

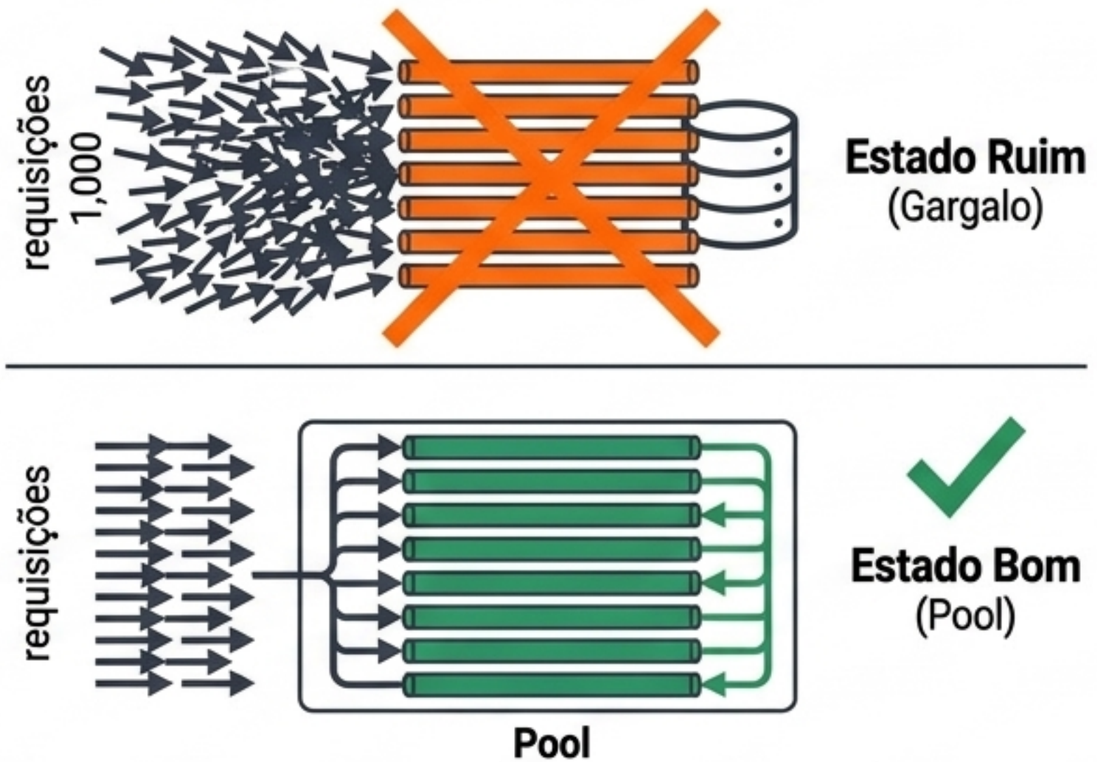
Configuração Segura e Desempenho

Variáveis de Ambiente (.env)



Blinda credenciais sensíveis, mantendo-as fora do código-fonte e longe de repositórios como o GitHub.

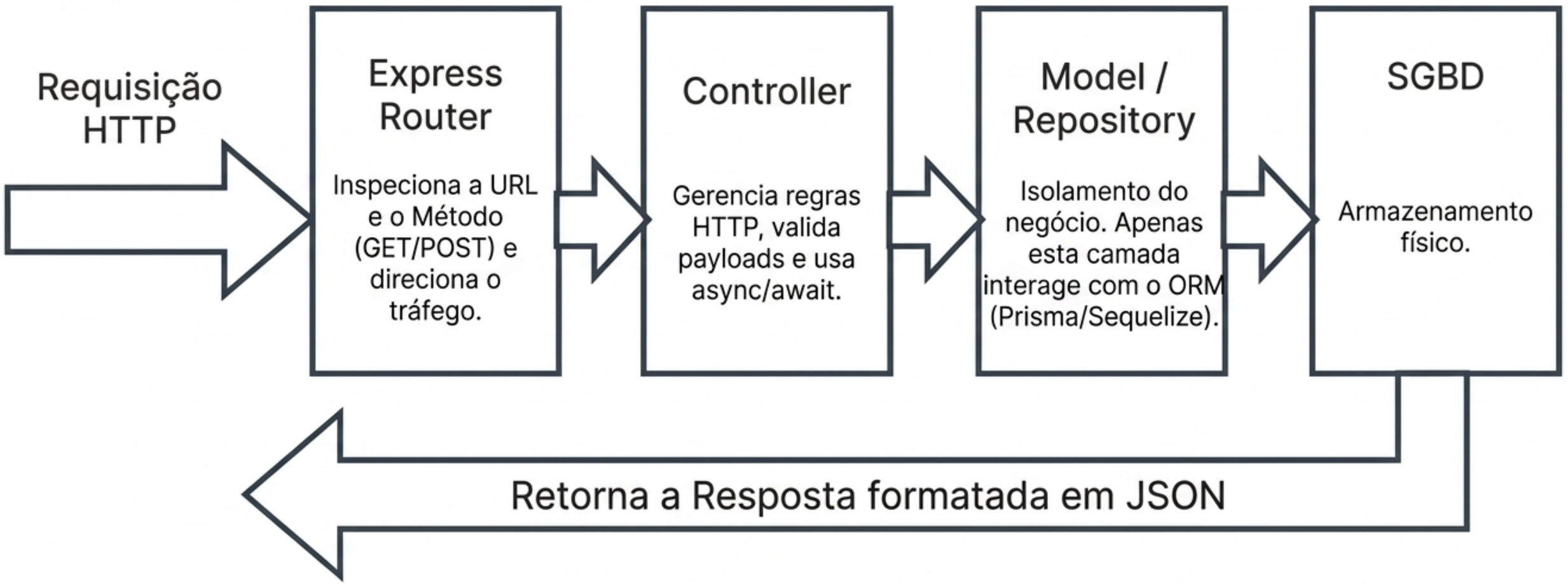
Connection Pooling



Em vez de abrir e fechar conexões constantemente (travando o V8 Event Loop), um Pool mantém conexões ativas que são 'emprestadas' para as requisições e devolvidas logo em seguida.

Gemini Notebook

Arquitetura MVC / Repository na Prática



Gemini Notebook

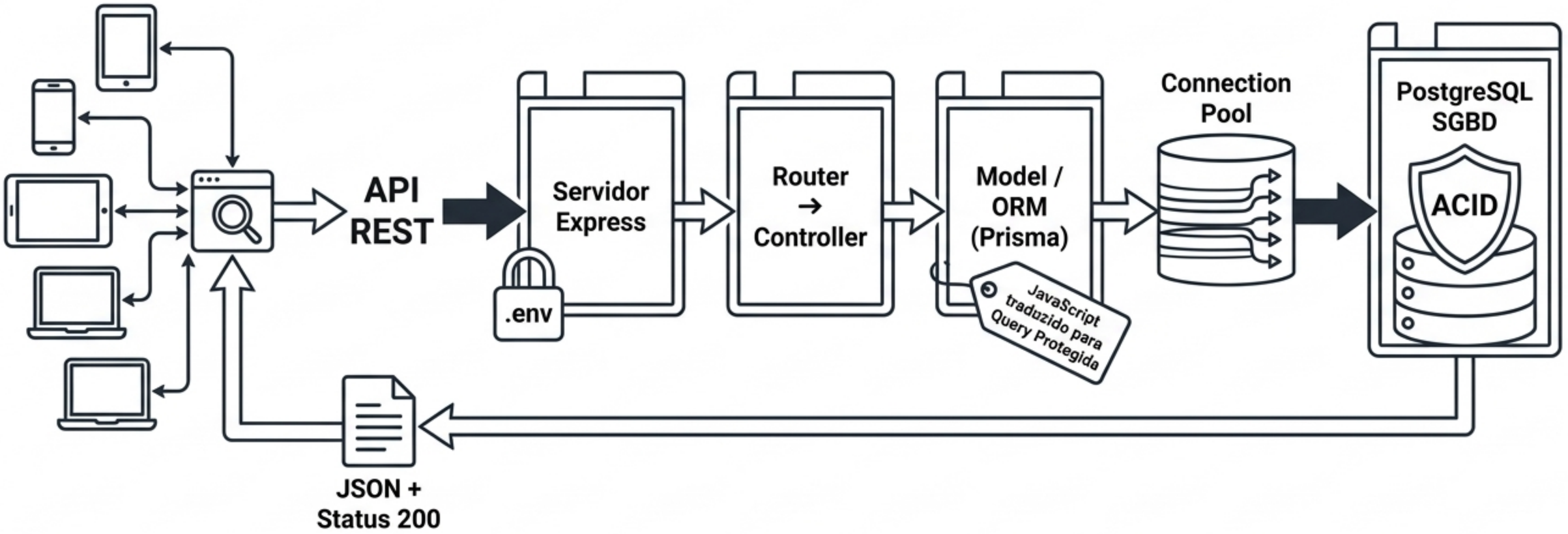
O Fluxo de uma Resposta HTTP Perfeita

Mapeamento semântico entre o Banco de Dados e a API REST

Cheatsheet	
200 OK	Busca (SELECT) no banco bem-sucedida ou alteração realizada.
201 Created	Inserção (INSERT) concluída com sucesso. Um novo registro foi gerado via ORM.
400 Bad Request	Falha na validação dos campos obrigatórios no Schema/Model antes mesmo de atingir o banco.
404 Not Found	A query (findOne) executou perfeitamente, mas retornou nulo. O registro solicitado não existe.
500 Internal Error	Falha catastrófica. O SGBD caiu, ocorreu timeout ou falha crítica no pool de conexões.

Gemini Notebook

Síntese Arquitetural: The Big Picture



A arquitetura web madura exige abstração para garantir escalabilidade, segurança e integridade. O back-end estruturado é o guardião inegociável dos dados em produção.

Gemini Notebook